

CASO PRÁCTICO · 2 SEMANAS

Tu propio sistema multi-agente

Diseña y construye un proyecto guiado por IA que integre orquestación multi-agente, skills + MCP y eficiencia de tokens.

Buenas prácticas de IA · Nivel intermedio · Puesta en común: Sesión 3

OBJETIVO

Integra los tres pilares del caso real

El entregable no es solo la app: es el sistema que permite a la IA generarla, revisarla y mantenerla de forma eficiente.

1

1 · Orquestación multi-agente

Un orquestador y un revisor adversarial con el bucle SPEC → PLAN → CODE → REVIEW y puntos de aprobación.

2

2 · Skills + herramientas (MCP)

≥ 1 skill reutilizable y ≥ 1 herramienta (MCP o mock) que la IA active según el contexto.

3

3 · Eficiencia de tokens

Reduce consumo y evita límites de uso sin perder calidad, con datos que lo demuestren.

ENUNCIADO

Elige un producto pequeño pero real

Un SaaS o herramienta interna, acotado para terminarlo en 2 semanas, pero lo bastante rico para justificar un bucle con revisión. Puedes proponer el tuyo.

● Clasificador / triage

Tickets de soporte, issues o correos entrantes.

● Extractor estructurado

De facturas, PDFs o logs a JSON validado.

● Generador de artefactos

Informes, changelogs o documentación a partir de datos.

● Mini front guiado por diseño

Una vista a partir de un componente o frame.

PILAR 1

Orquestación multi-agente

Reproduce, a tu escala, el patrón del caso real: un orquestador que dirige y un auditor que revisa de forma adversarial.

Orquestador

Dirige la tubería de principio a fin.

- Encadena SPEC → PLAN → CODE → REVIEW → COMMIT.
- Genera tareas y decide cuándo llamar al revisor.
- Interactúa contigo en los gates de cada fase.

Auditor (revisor)

Critica, no escribe la solución.

- Evalúa los diffs contra reglas explícitas (rules.md).
- Emite veredicto PASS / WARNINGS / FAIL.
- Bucle con reanudación: cero trabajo repetido.

PILAR 2

Skills y herramientas (MCP)



≥ 1 skill reutilizable

Encapsula una tarea recurrente: instrucciones + pasos + ejemplo, para que la IA la aplique de forma consistente. Idealmente auto-activable según el contexto (como fe-architect, ui-kit o figma-sync del caso real).



≥ 1 herramienta

Un servidor MCP o una mock API que reemplace una dependencia externa, de modo que el sistema funcione y se pueda testear sin credenciales reales.

PILAR 3

Eficiencia de tokens (y se mide)

Selección de modelo por tarea

Ligero para lo repetitivo, equilibrado para codificar/revisar, potente solo para orquestar. Justifica cada elección.

Gestión del contexto

Sesiones cortas, /clear y /compact, contexto preciso (@file) y specs/planes persistidos para reanudar.

Subagentes

Aísla trabajo pesado (búsqueda amplia, verificación) para no contaminar el contexto principal.

Medición

Línea base 'ingenua' vs. flujo optimizado: reporta tokens y/o nº de turnos/iteraciones.

RESTRICCIÓN TRANSVERSAL

Nada de vibe coding: define el contrato

Antes de generar código, define el contrato que guía a la IA. Si está bien definido, el código será coherente; si falta, la IA improvisa.

SPEC .md**EL QUÉ**

Problema, usuarios, casos de uso, criterios de aceptación y restricciones de negocio.

ARCHITECTURE .md**EL CÓMO**

Componentes, flujos de datos, stack, decisiones y trade-offs (incl. modelos por tarea).

AGENTS .md**LAS INSTRUCCIONES**

Cómo construir, testear y validar; convenciones, comandos y reglas de los gates.

ENTREGABLE

Qué entregar

Un único repositorio con dos cosas: la app y el sistema que la genera y la gobierna (esto último es lo que más pesa).

- **La app + el sistema que la genera** — es lo que más pesa en la evaluación.
- **Sistema de especificaciones** — SPEC.md (qué) · ARCHITECTURE.md (cómo) · AGENTS.md (instrucciones).
- **Configuración de arquitectura por skill** — instrucciones, comandos y ejemplos (auto-activables).
- **Herramientas** — ≥ 1 MCP o mock API que sustituya dependencias externas.
- **Pipeline multi-agente con orquestación** — orquestador + auditor, bucle SPEC→PLAN→CODE→REVIEW→COMMIT con comandos, gates y rules.md.
- **Informe de eficiencia** — modelo por tarea y tokens antes/después (EFFICIENCY.md).

ENTREGABLE

Estructura sugerida del repositorio

```
mi-proyecto/
├── README.md           # Instalar, usar y reproducir con IA
├── SPEC.md             # Qué hace el producto
├── ARCHITECTURE.md     # Cómo está construido (modelos por tarea)
├── AGENTS.md           # Instrucciones para los agentes
├── agents/
│   ├── orchestrator.md # Rol, modelo, bucle y gates
│   └── reviewer.md     # Rol, modelo y referencia a rules.md
├── rules.md            # Reglas del auditor (PASS/WARN/FAIL)
├── skills/<mi-skill>.md # Skill reutilizable (auto-activable)
├── tools/<mock-o-mcp>.* # Mock API o servidor MCP
├── src/                # La app
├── tests/golden.jsonl  # Dataset de validación (si aplica)
└── EFFICIENCY.md       # Tokens: línea base vs. optimizado
```

EVALUACIÓN

Cómo se evalúa

30%

Sistema generable

Que otra persona regenere la app dándole SPEC + ARCHITECTURE + AGENTS.

25%

Orquestación + revisión

Bucle completo, gates útiles y un auditor que detecta y fuerza correcciones.

20%

Eficiencia de tokens

Reducción demostrada con datos; modelo y contexto justificados.

15%

Skills + herramientas

Reutilización real; skill bien encapsulada y herramienta/MCP funcional.

10%

Trazabilidad y calidad

Planes y revisiones persistidos; código coherente con la spec.



Recomendaciones

- Empieza por el contrato (SPEC/ARCHITECTURE/AGENTS), no por el código.
- Mantén el alcance pequeño: mejor un bucle completo y medido que una app grande sin sistema.
- Mide los tokens desde el primer día: es lo que más diferencia tu entrega.
- Captura lo que funcione como skill para no repetirte.

Entrega: repositorio (GitLab de GFT) antes de la Sesión 3 · Dudas, en el canal del curso.